Questo documento è basato sulla dispensa della docente Anna Corazza,su contenuti presi dal web e su altri riassunti fatti da altri studenti di cui non si ha il nome.

Protocollo HTTP

L' HTTP è un protocollo di Internet atto alla gestione della comunicazione di due entità remote.Una caratteristica importantissima del protocollo HTTP è di essere senza stato (stateless), né il server né il client mantengono informazioni sullo stato della comunicazione. Questo significa che per mantenere delle informazioni sullo stato della comunicazione, bisogna implementare dei meccanismi che lo facciano;

Esistono due tipi di messaggi HTTP:

1) messaggio di richiesta (client -> server)

2) messaggio di risposta (server -> client)

Composizione messaggio HTTP:

composizione base:
1) una prima riga che prende il nome di **request line** nei messaggi di richiesta, o **status line** nei messaggi di risposta.
2) **Headers**: delle righe di intestazione (anche nessuna)
3) Una riga vuota
4) **Body:** Il corpo del messaggio

La request line ha una struttura di questo tipo:

CAMPO METODO - CAMPO URL (OGGETTO RICHIESTO) - VERSIONE HTTP

```
GET /hello.htm HTTP/1.1
User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)
Host: www.tutorialspoint.com
Accept-Language: en-us
Accept-Encoding: gzip, deflate
Connection: Keep-Alive
```

Come funziona il caching in HTTP?

Il caching è un meccanismo importante in HTTP che consente di memorizzare temporaneamente le risorse sul client o su server intermedi (come proxy o cache) per ridurre il tempo di risposta e il carico sul server. Quando un client effettua una

richiesta HTTP, il server può includere nell'header di risposta l'indicazione che il contenuto può essere memorizzato nella cache del client o dei server intermedi. Successivamente, se viene effettuata una richiesta identica per la stessa risorsa, il client o il server intermedio può restituire il contenuto memorizzato in cache anziché richiederlo nuovamente al server originale. Ciò riduce la latenza e il traffico di rete. Il caching può essere controllato utilizzando gli header HTTP come "Cache-Control" ,"Expires", “If-Modified-Since”,etc.

Cookies

Un cookie web è un piccolo file di testo che è strutturato in modo specifico e contiene diverse informazioni,esso può servire per vari scopi.

Ogni cookie viene sempre creato dal server, che lo passa al client nel campo setcookie dell’intestazione della risposta HTTP. Il client può decidere di salvarlo. Se il cookie è già stato inviato dal server, c'è un meccanismo che evita di far rinviare al server nuovamente il cookie, specificando nella richiesta GET da parte del client. Se il server vuole che un cookie venga cancellato, ne invia quindi uno con lo stesso nome, host e path di quello da cancellare, ma con expiration date negativa, in modo che quest’ultimo venga prima sovrascritto e immediatamente cancellato.

Ecco come è solitamente strutturato un cookie web in dettaglio:

1. **Nome del cookie**: Questo è il nome del cookie e viene utilizzato per identificarlo in modo univoco. È un campo obbligatorio. Ad esempio, "username" potrebbe essere il nome di un cookie che memorizza il nome utente dell'utente.

2. **Valore del cookie**: Questo è il contenuto del cookie, cioè le informazioni effettive che il cookie memorizza. Ad esempio, se il cookie è "username," il suo valore potrebbe essere "JohnDoe"

3. **Dominio e percorso associati**: Puoi specificare il dominio e il percorso del sito web a cui il cookie è associato. Questo determina su quali pagine del sito web il cookie verrà inviato. Ad esempio, un cookie potrebbe essere associato al dominio "example.com" e al percorso "/shop," il che significa che verrà inviato solo quando l'utente visita le pagine del negozio su "example.com."

4. **Scadenza del cookie**: Puoi specificare quando il cookie scadrà e verrà automaticamente rimosso dal dispositivo dell'utente. Questo può essere una data e un'ora specifiche o un periodo di tempo a partire dalla creazione del cookie. Ad esempio, un cookie potrebbe scadere il 1º gennaio 2024.

```
Nome del cookie: username
Valore del cookie: JohnDoe
Dominio: example.com
Percorso: /
Scadenza: 1 gennaio 2024
Sicurezza: Sicuro
HttpOnly: Sì
```

I cookies sono utilizzati dai server web per memorizzare informazioni e comunicarle al browser dell'utente durante le visite successive al sito. Queste informazioni possono essere utilizzate per autenticare gli utenti, tracciare le sessioni di navigazione, personalizzare l'esperienza dell'utente e altro ancora.

Session Tracking

Il session tracking è un concetto utilizzato principalmente nel contesto dello sviluppo web e si riferisce alla pratica di monitorare e gestire le sessioni degli utenti durante la loro interazione con un'applicazione web. Una sessione in questo contesto rappresenta un periodo di tempo in cui un utente interagisce con un'applicazione web specifica, come un sito web o un'applicazione web.

Ecco alcune informazioni chiave sul session tracking:

approfondire?

1. **Scopo del Session Tracking**: Il session tracking serve a mantenere uno stato di "connessione" tra il server web e il client (solitamente un browser web) durante la navigazione dell'utente. Questo stato consente all'applicazione di tenere traccia delle azioni e delle interazioni dell'utente nel corso di una sessione, consentendo così di offrire una migliore esperienza utente e di personalizzare il contenuto in base alle azioni dell'utente.
2. **Implementazione del Session Tracking**: Il session tracking può essere attuato in diversi modi. Alcuni dei metodi più comuni includono l'uso di cookie, URL rewriting, nascosto (hidden) input fields in un modulo HTML e la gestione dei dati di sessione lato server. Di seguito sono descritti brevemente alcuni di questi metodi:

    - **Cookie**: I cookie sono piccoli file di testo memorizzati sul computer dell'utente. Possono essere utilizzati per mantenere un identificatore di sessione univoco, consentendo al server di associare richieste successive dello stesso utente alla sua sessione.

- **URL Rewriting**: Questo metodo implica l'inclusione dell'identificatore di sessione direttamente nell'URL. Ad esempio, `http://www.example.com/index.php?sessionID=12345`. Ogni volta che l'utente fa una richiesta, l'identificatore di sessione viene estratto dall'URL.
- **Hidden Input Fields**: In un modulo HTML, è possibile includere campi nascosti contenenti l'identificatore di sessione. Quando il modulo viene inviato, l'identificatore di sessione viene inviato al server.
- **Gestione lato server**: Alcuni linguaggi di programmazione web, come PHP o Java, offrono funzionalità di gestione delle sessioni. Questo metodo memorizza i dati di sessione lato server e assegna un cookie all'utente per identificare in modo univoco la sessione.

HTML

HTML (Hyper-Text Mark-Up Language) è un linguaggio di mark-up (di etichettatura) usato principalmente per le pagine web; esso tramite l'utilizzo di tag (etichette del tipo <**tag**>) va e definire gli elementi della pagina web che vengono poi renderizzati da un client web , per esempio un browser.
In altre parole, HTML è il linguaggio di base utilizzato per progettare e formattare le pagine web, consentendo di definire la struttura logica e il layout visuale del contenuto web.
Ogni tag ha una funzione specifica e può essere utilizzato per creare titoli, paragrafi, elenchi, link ipertestuali, immagini, form e molto altro. I browser web interpretano il codice HTML e visualizzano il contenuto della pagina in modo appropriato per l'utente.

Le diverse versioni di HTML sono state sviluppate nel corso degli anni per migliorare la funzionalità e l'efficienza del linguaggio. Le versioni principali di HTML includono:

1. HTML 2.0: Questa è stata la prima versione standardizzata di HTML, introdotta nel 1995.
2. HTML 3.2: Questa versione è stata pubblicata nel 1997 e ha introdotto nuovi elementi e attributi.
3. HTML 4.01: Rilasciata nel 1999, questa versione ha portato ulteriori miglioramenti, come il supporto per fogli di stile CSS.
4. XHTML 1.0: Pubblicata nel 2000, XHTML (eXtensible HyperText Markup Language) è una riformulazione di HTML 4.01 in XML, il che significa che segue una sintassi più rigorosa.

5. HTML5: Questa è la versione più recente e significativa di HTML, introdotta nel 2014. HTML5 ha portato molte nuove funzionalità e miglioramenti, tra cui il supporto per l'audio e il video integrati, il trascinamento e il rilascio, il canvas per la grafica interattiva e molto altro. HTML5 è molto più flessibile ed è progettato per supportare una vasta gamma di dispositivi e scenari di utilizzo, inclusi dispositivi mobili.

HTML5 è attualmente la versione predominante e più utilizzata di HTML, ed è ampiamente supportata da tutti i principali browser web. Molte delle caratteristiche avanzate di HTML5 hanno reso possibile la creazione di applicazioni web avanzate e interattive.

Quali sono le principali differenze tra le vecchie versioni di HTML e HTML5?

HTML5 ha effettuato una pulizia del linguaggio eliminando o deprecando alcuni elementi obsoleti o poco utilizzati riscontrati nelle versioni precedenti di HTML. Tra gli elementi deprecati in HTML5 ci sono i tag relativi alle applet oramai obsolete e non più supportate da nessun browser web,e tutti i tag relativi alla formattazione della pagina e il suo stile per il quale si deve adesso prediligere CSS.
Abbiamo poi inoltre audio e video nativi, più interazione generale con javascript, API di geolocalizzazione,API di storage (session storage e local storage), e nuovi tag.

Le API di Web Storage in HTML5 sono un insieme di metodi e oggetti che consentono di memorizzare dati in modo persistente sul lato client (cioè sul computer dell'utente) e di accedervi tramite JavaScript. Queste API forniscono due meccanismi principali di memorizzazione dei dati: `localStorage` e `sessionStorage`.

Ecco una panoramica delle due API di Web Storage:

1. **localStorage**

- **localStorage** consente di memorizzare dati in modo persistente nel browser dell'utente, in modo che i dati siano disponibili anche dopo la chiusura del browser e il riavvio del computer.

- I dati memorizzati con **localStorage** sono associati a un dominio web specifico e restano disponibili fino a quando l'utente o il sito web non li elimina esplicitamente.

- I dati sono memorizzati come coppie chiave-valore, simili ai cookie, ma con una capacità di archiviazione notevolmente maggiore (solitamente alcuni megabyte).

- Gli oggetti memorizzati devono essere convertibili in stringhe JSON.

Esempio di utilizzo di **localStorage**:

```
localStorage.setItem("nome", "John");
// Recuperare un valore
var nome = localStorage.getItem("nome");
// Rimuovere un valore
localStorage.removeItem("nome");
```

2. **sessionStorage**:

- **sessionStorage** è simile a **localStorage**, ma memorizza i dati solo per la durata della sessione del browser.

- I dati memorizzati con **sessionStorage** sono disponibili solo per la durata della sessione attuale. Quando il browser viene chiuso o la scheda viene chiusa, i dati vengono cancellati automaticamente.

- Anche **sessionStorage** utilizza il formato chiave-valore e i dati devono essere convertibili in stringhe JSON.

Esempio di utilizzo di **sessionStorage**:

```
// Salvare un valore
sessionStorage.setItem("lingua", "italiano");
// Recuperare un valore
var lingua = sessionStorage.getItem("lingua");
// Rimuovere un valore
sessionStorage.removeItem("lingua");
```

Le API di Web Storage sono utili per la memorizzazione temporanea di dati sul lato client, come preferenze utente, dati di sessione, informazioni di configurazione e molto altro. Tuttavia, è importante notare che questi dati sono accessibili da JavaScript, quindi non dovrebbero essere utilizzati per memorizzare informazioni sensibili o critiche per la sicurezza.

Problemi di sicurezza di HTML

1) **Same-origin policy**

Ad uno user agent quasi sempre viene richiesto di eseguire script, ovviamente potrebbero esserci dei problemi nel caso in cui uno di questi script sia malevolo. In particolare si potrebbe correre il rischio di dare accesso ad informazioni riservate fornite da altri siti. Ovviamente occorre cautelarsi.per affrontare questo problema, gli user agent usano il concetto di origine per implementare la cosiddetta same-origin policy. Il principio seguito è che i contenuti possono interagire senza restrizione con tutti i contenuti provenienti dalla stessa origine, mentre questo non può avvenire tra contenuti di origine diversa.

2) Cross-site scripting

Nel momento in cui si introducono delle form in un documento HTML è importante fare attenzione al testo che viene inserito. Occorre quindi inserire filtri per la validazione dei dati in ingresso, precisamente filtri basati sull'enumerazione dei casi favorevoli (whitelist-based). L'attacco consiste nel convincere l'utente a cliccare su un link che fa partire uno script creato dall'attaccante allo scopo di danneggiare l'utente.

3) **Cross-Site Request Forgery (CSRF)**

Nel momento in cui si utilizzano delle form si può incorrere in alcune situazioni pericolose, come il fatto che l'utente possa compiere azioni non intenzionalmente. L’origine di questo problema risiede nel fatto che HTML permette all’utente di sottomettere le form anche a origini diverse da quelle dell’applicazione. La prevenzione di questo tipo di attacchi richiede di popolare le form con token nascosti caratterizzanti l’utente oppure di controllare il campo Origin nell’intestazione di tutte le richieste HTTP.

4) **Clickjacking**

Un altro problema di sicurezza comune è il Clickjacking, ovvero durante una normale navigazione web, l'utente clicca con il puntatore del mouse su di un oggetto (ad esempio un link), ma in realtà il suo clic viene reindirizzato, a sua insaputa, su di un altro oggetto, che può portare alle più svariate conseguenze: dal semplice invio di spam, al download di un file, fino ad ordinare prodotti da siti di e-commerce. Oltre al rapimento del clic, possono essere intercettati i tasti premuti, ad esempio quando si inserisce la password dell'email o del conto bancario. Entrambe queste possibilità (rapimento del clic o dei tasti premuti sulla tastiera) vengono eseguite senza la cognizione dell'utente.

JAVA

Il linguaggio Java è particolarmente adatto alle applicazioni web per molteplici motivi.

1)E' un linguaggio orientato agli oggetti e questo lo rende particolarmente semplice

2)Ha una gestione automatica della memoria che semplifica le operazioni di deallocazione della memoria.

3)E' fortemente tipato ed evita molteplici errori, molto robusto (questo perchè java è molto stretto sui tipi, tant'è che il type-checking avviene a tempo di compilazione).

4)Non ammette costrutti insicuri quali un accesso a vettore senza controllo sugli indici, perché questo tipo di costrutti possono dare origine a comportamenti del programma non specificati e non prevedibili.

5)E' indipendente dalla piattaforma perché viene compilato in bytecode ed eseguito su una Java Virtual Machine.

6)Possiede un buon supporto al multi-thread.

7)E' particolarmente sicuro per le applicazioni web. La Java Virtual Machine è una macchina astratta e non vengono fatte ipotesi sulla particolare tecnologia in cui viene implementata né sulla piattaforma ospite.Non sa nulla del linguaggio Java, ma solo del formato class (bytecode). I bytecode possono anche venir compilati in file eseguibili dalle diverse macchine Una piattaforma Java è composta da:

• Linguaggio Java;
• Java Virtual Machine (JVM);
• Application Programming Interfaces (API libraries).

La sicurezza del linguaggio java è determinata da:

Verifica del bytecode prima dell'esecuzione;
mentre durante l'esecuzione:
• Le classi vengono caricate e ne viene fatto il linking;
• Opzionale: generazione del codice macchina e ottimizzazione dinamica;
• Esecuzione del programma: durante questo processo, il class loader definisce un namespace locale, in modo che codice non sicuro non possa interferire con l'esecuzione di altri programmi Java.
• La mediazione da parte della Java Virtual Machine mediante una classe security manager per l'accesso alle risorse di sistema.

Alcune delle caratteristiche rendono molto sicuro il linguaggio, rendendo impossibili certi tipi di azioni: ad esempio, l'**assenza di aritmetica dei puntatori**. Vietare l'aritmetica dei puntatori non equivale a non supportare una qualche forma di puntatori, che in Java sono chiamati references e pienamente supportati ma impedisce al programmatore di creare codice poco sicuro, che accede a celle di memorie vietate etc.

Applets

Si tratta di uno strumento per la programmazione lato client basato su Java. Le applet Java sono programmi scritti in linguaggio Java che possono essere eseguiti da un web browser. Le Java applets sono eseguibili dai web browser che utilizzano la Java virtual machine (JVM). Per eseguirne il contenuto, la maggioranza dei Web Browser utilizza una sandbox, in modo da impedire alle applet di accedere alle informazioni salvate in locale sul computer. Il codice sorgente delle Applet viene scaricato dal web server attraverso il web browser ricevendo anche la pagina html che lo contiene. Le Applet mostrate nelle pagine web sono identificate dal tag html ... (HTML vecchia versione), al quale è preferibile (HTML 4.0).
Dato che il bytecode Java è indipendente dalla piattaforma, gli Applet Java possono essere eseguiti senza problemi dai browser delle principali piattaforme.Una applet di fatto è un'applicazione priva di main. le applet devono gestire gli eventi, quali i click del mouse, bottoni, campi in cui l'utente può introdurre testo, etc.. I browser hanno in genere un **SecurityManager** (classe java che permette di definire delle regole di sicurezza nella JVM) . Le politiche di sicurezza previste nei browser prevedono che applet scaricate dalla rete:
• non possono leggere o scrivere file sulla macchina ospite;
• non possono creare connessioni di rete con host diversi dal server da cui sono stati scaricati;
• non possano caricare librerie o essere composte da metodi nativi (possono chiamare metodi nativi, ma non averli al proprio interno);
• non possono avviare nuovi processi sulla macchina ospite;
• non possono avere accesso a informazioni sulla macchina locale, tranne la versione di Java in uso, il nome e la versione del sistema operativo e alcune informazioni sui 8 caratteri utilizzati; in particolare, non possono accedere al nome utente, al suo indirizzo di posta elettronica, etc.;
• tutte le finestre aperte da un applet visualizzano un messaggio di avvertimento. Scaricare le applet è relativamente costoso, anche perché occorre una richiesta diversa per ogni classe. Per questo motivo risulta conveniente impacchettare tutti i file .class ed eventuali immagini e/o suoni in un unico file JAR (Java ARchive) compresso che può essere scaricato con una sola richiesta. il ciclo di vita delle applet è definito da 4 metodi della classe Applet:
1) **init(**) : viene chiamato automaticamente dal sistema quando l'applet comincia l'esecuzione, comprende quindi tutte le inizializzazioni di immagini,oggetti ecc.
2) **start()**: viene chiamato dopo init() e ogni volta che l'utente torna alla pagina contenente l'applet; quindi, mentre init() viene chiamato una sola volta, start() può venir chiamato più volte; di solito è da qui che viene riavviato il thread, ad esempio per riprendere l'animazione; non è necessario implementarlo (in coppia con stop()), se non c'è alcuna operazione che debba essere sospesa quando l'utente esce dalla pagina corrente;
3) **stop(**): non è necessario richiamarlo esplicitamente, in quanto viene richiamato ogni volta che l'utente esce dalla pagina in cui si trova l'applet; serve a sospendere tutte quelle attività che consumerebbero risorse inutilmente;

4) **destroy()**: viene richiamato quando il browser viene chiuso normalmente, dopo aver chiamato **stop()**; non occorre distruggere il pannello. è anche possibile passare dei parametri di ingresso all'applet, usando l'etichetta HTML PARAM, con attributi definiti dal programmatore. I parametri sono sempre restituiti come stringhe, che devono poi, ove necessario, essere convertite nel tipo richiesto. Il nome dell'attributo deve corrispondere esattamente al parametro passato al metodo getParameter, maiuscole e minuscole comprese.

Servlet

Le Servlet sono degli strumenti utilizzati per la programmazione Java lato-server, esse servono infatti a gestire le richieste dei client web e rispondere a queste richieste.
Esse presentano tutti i vantaggi della programmazione Java già discussi, e cosa più importante è che supportano il **multi-threading** (cosa che ad esempio le CGI non facevano, creando un processo per ogni richiesta).
D'altra parte, però, occorre che il server sia predisposto all'esecuzione delle servlet, e preveda quello che si chiama **servlet engine** o **servlet container,** come ad esempio Apache Tomcat.
Le **servlet** sono delle classi Java che possono venir caricate dinamicamente per estendere le funzionalità del server. A differenza delle estensioni proprietarie, le servlet vengono eseguite all'interno della Java Virtual Machine sul server, e sono quindi sicure e portabili.
Inoltre le servlet offrono un'ottima portabilità sia su sistemi operativi diversi che su server e servlet engines diversi. Oltre alle servlets vere e proprie, esistono le Java Server Pages (JSP) che vengono tradotte in servlets e forniscono quindi un sistema per implementare il meccanismo di richiesta e risposta del server senza doversi occupare di tutti i dettagli implementativi delle servlets.
Normalmente, si usano:
• le **JSP** quando la maggior parte del contenuto passato al client è testo statico e markup, mentre solo una piccola porzione del contenuto viene prodotta dinamicamente mediante codice Java;
• le **servlets** quando la porzione di testo da produrre staticamente è poca; infatti, la maggior parte delle servlets non produce testo, ma porta a termine dei compiti richiesti dal client, e alla fine richiama altre servlets o JSPs per produrre una risposta. Le servlet normalmente passano il loro risultato al client sotto forma di file HTML, XHTML, XML da visualizzare nel browser, ma anche in altri formati, quali immagini e dati binari. Anche se vengono tipicamente usate per server Web, le servlet possono estendere ogni tipo di server: di posta, FTP, etc.. Infatti, tutte le servlet devono implementare l'interfaccia Servlet, che è la classica API di un fornitore di servizi.

I metodi dell'interfaccia Servlet vengono invocati automaticamente dal servlet engine.

```
interface Servlet
{
public void init(ServletConfig config) throws ServletException;
public ServletConfig getServletConfig();
public void service(ServletRequest req, ServletResponse res) throws
ServletException, IOException;
```

```
public String getServletInfo(); public void destroy();
}
```

L'interfaccia dichiara quindi cinque metodi, di cui però non dà il body e quindi l'implementazione essendo appunto un interfaccia:
1. void **init**(ServletConfig config): inizializza la servlet una volta sola durante il ciclo di esecuzione della servlet;
2. ServletConfig **getServletConfig()**: ritorna un oggetto che implementa l'interfaccia ServletConfig e fornisce l'accesso alle informazioni sulla configurazione della servlet, tra cui i parametri di inizializzazione e il ServletContext, ovvero il suo ambiente (il servlet engine in cui vengono eseguite);
3. String **getServletInfo()**: viene definito in modo da restituire una stringa con informazioni quali autore e versione;
4. void **service(ServletRequest request, ServletResponse response)**: viene mandato in esecuzione in risposta alla richiesta mandata da un client alla servlet;
5. void **destroy()**: invocato quando la servlet viene terminata; usato per rilasciare le risorse (connessioni a basi di dati, file aperti, etc.).
Il metodo **init()** viene invocato solo quando la servlet viene caricata in memoria, di solito a seguito della prima richiesta per quella servlet. Solo dopo che il metodo è stato eseguito, il server può soddisfare la richiesta del client invocando il metodo service, che riceve la richiesta, la elabora e prepara la risposta per il client. Quindi il metodo service viene richiamato ogni volta che si riceve una richiesta. Tipicamente, per ogni nuova richiesta il servlet engine crea una nuova thread di esecuzione in cui viene mandato in esecuzione service. Solo quando il servlet engine termina la servlet, viene invocato il metodo destroy per rilasciare le risorse della servlet.

I pacchetti delle servlet definiscono due classi astratte che implementano l'interfaccia Servlet:
• GenericServlet (in javax.servlet);
• HttpServlet (in javax.servlet.http).
Queste classi forniscono un'implementazione di default per tutti i metodi dell'interfaccia. A noi interessa la classe HttpServlet:

```
public abstract class HttpServlet extends GenericServlet
{
      protected void doGet(HttpServletRequest req, HttpServletResponse resp)
      throws ServletException, IOException)
      {...}
      protected void doPost(HttpServletRequest req, HttpServletResponse resp)
      throws ServletException, IOException)
      {...}
      protected void doPut(HttpServletRequest req, HttpServletResponse resp)
      throws ServletException, IOException)
      {...}
      protected void doDelete(HttpServletRequest req, HttpServletResponse resp)
      throws ServletException, IOException)
```

che implementa una servlet rispetto al protocollo HTTP, e quindi per il web. Essa fornisce una classe astratta che verrà estesa dall'implementazione delle classi che lo sviluppatore implementa per realizzare la propria applicazione. Ogni sottoclasse di HttpServlet deve fare overriding di almeno un metodo, di solito uno dei seguenti:
• doGet(), se la servlet viene richiamata col metodo HTTP GET;
• doPost(), per richieste HTTP con metodo POST;
• doPut(), per richieste HTTP con metodo PUT;
• doDelete(), per richieste HTTP con metodo DELETE;
• init() e destroy(), per gestire eventuali risorse necessarie al funzionamento della servlet (ad esempio, connessioni con la base di dati);
• getServletInfo(), usata dalla servlet per fornire informazioni riguardanti se stessa.
Una strategia tipicamente utilizzata è di non riscrivere il metodo service, ma piuttosto i metodi "doXXX". La maggior parte delle servlet estende una di queste due classi e sovrascrive (override) uno o più dei suoi metodi. In applicazioni Web di solito si usano sottoclassi di HttpServlet. Il metodo più significativo in una servlet è ovviamente il metodo service, o in alternativa di metodi doXXX, che prendono in ingresso:

• un oggetto ServletRequest, corrispondente ad uno stream di ingresso, da cui leggere la richiesta del client;
• un oggetto ServletResponse, corrispondente ad uno stream di uscita, su cui scrivere la risposta. Questi stream possono essere basati su byte o su caratteri.
In caso di problemi, viene lanciata una ServletException o una IOException. In caso di richieste multiple, diverse chiamate al metodo service verranno eseguite in parallelo: può quindi rendersi necessario che i programmatori serializzino l'accesso alle risorse mediante tecniche di sincronizzazione.

N.B.
La professoressa nella sua dispensa mostra vari esempi di implementazione di servlet tramite HttpServlet, tra i quali l'utilizzo del metodo service in alternativa ai vari doXXX; la documentazione della classe sul sito di Oracle ci dice tuttavia che implementare service per gestire le richieste HTTP risulta inutile in quanto questo metodo chiamerà a sua volta gli handler doXXX definiti sopra.

CMS e Framework

Contesto RIA e Web 2.0

Le moderne applicazioni per il web hanno molte caratteristiche in comune con le classiche applicazioni desktop, sia in termini tecnologici, che in termini di esperienza e interazione utente, tanto da guadagnarsi la denominazione di Rich Internet Applications (**RIA**). Le RIA, infatti, utilizzano tecnologie lato client in maniera intensiva, quali Javascript e CSS3 in combinazione con AJAX per l'invio asincrono di richieste HTTP, per la realizzazione di interfacce utente sempre più simili a quelle desktop.

Per tali ragioni, lo sviluppo e la diffusione delle RIA hanno sancito la nascita del cosiddetto Web 2.0, un contesto dove appunto i siti web sono delle vere e proprie applicazioni che coinvolgono l'utente e gli permettono di effettuare una miriadi di operazioni differenti a seconda dei servizi offerti. In questo contesto vanno considerati i Content Management Systems (CMS) ed i Web Application Frameworks o, più brevemente, Web Frameworks, entrambi discussi in questo capitolo. I CMS hanno lo scopo di gestire workflows e lo sviluppo di contenuti in ambienti di lavoro collaborativo e vengono quindi progettati per semplificare la pubblicazione di documenti su web. Alcuni dei più noti e più utilizzati CMS sono Joomla, Drupal, WordPress. I Web Frameworks, invece, si prefiggono lo scopo di alleggerire il lavoro legato allo sviluppo di applicazioni web, fornendo supporto per le funzionalità tipicamente richieste per la loro realizzazione quali, ad esempio, la gestione e la visualizzazione dei dati. Attualmente esiste un notevole numero di web frameworks, i quali si differenziano principalmente per le funzionalità di supporto messe a disposizione dello sviluppatore e, soprattutto, per il linguaggio di programmazione in cui essi sono implementati. Tra i web frameworks più noti si possono citare: **Apache Struts** e Play, basati su Java; **Ruby on Rails** basato su Ruby; CakePHP e Symfony basati su PHP; **Django** e Google App Engine basati su Python. Con riferimento alle RIA, infine, è possibile enfatizzare la differenza che sussiste tra CMS e web frameworks: i CMS sono RIA, mentre i web frameworks mettono a disposizione gli strumenti per la sviluppo delle RIA.

Content Management Systems (CMS)

Come si evince anche dal nome i CMS sono dei sistemi per la gestione dei contenuti, che possono essere contenuti web o contenuti di altro genere (multimediali ad esempio) ;nel nostro caso ci focalizziamo sui Web CMS che sono essenzialmente quelli usabili da browser (ad esempio WordPress,Wikipedia) e che possono permettere (e non,dipende dal cms) con estrema semplicità d'uso funzioni del tipo:la creazione di contenuti da parte degli utenti, retrieval di informazioni su una base dati ad essa associata, management della base dati stessa e del pool di utenti, separazione degli accessi, facilità di personalizzazione, funzioni quali la modifica concorrente dello stesso documento da parte di utenti diversi etc.

Web Framework

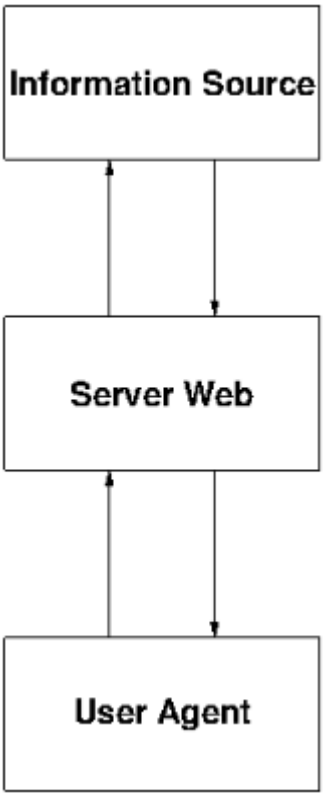
I web framework sono un insieme di strumenti che servono a semplificare lo sviluppo di applicazioni web, sia lato server che lato client.
Essi risultano molto utili nel ridurre sensibilmente i tempi di sviluppo in quanto semplificano quelle che sono le operazioni più comuni (che ogni sviluppatore deve ripetere ogni volta) come ad esempio la connessione ad una base dati, gestione delle richieste http da ambo le parti, etc.
Essi non solo velocizzano lo sviluppo, ma vanno anche a ridurre l'indice di errori che possono essere commessi e talvolta vanno anche a definire una architettura precisa

che uno sviluppatore dovrà seguire.
In questo modo lo sviluppatore non dovrà soffermarsi ampiamente sui tecnicismi e potrà quindi indirizzare la maggioranza del suo lavoro sullo sviluppo delle specifiche.

Architettura delle applicazioni web

Un'architettura (software) descrive la struttura di un'applicazione, la decomposizione in componenti, le loro interfacce e interazioni. L'architettura rappresenta il passaggio tra analisi e implementazione. Le applicazioni Web sono un esempio di architettura multi-tier (multistrato) o n-tier. Per tier si intende un raggruppamento logico di funzionalità. I diversi tier possono stare sulla stessa macchina o su macchine diverse.

Le applicazioni Web più semplici sono a tre tier, quindi three-tier, come quella illustrata:



Il tier intermedio implementa meccanismi che permettono la comunicazione tra il client e la sorgente dell'informazione tramite:

• il meccanismo di controllo (**controller logic**): elabora la richiesta del client e si procura i dati da presentargli;

• il meccanismo di presentazione (**presentation logic)** elabora questi dati per presentare all'utente (in HTML, XHTML, WML, ...)

• le regole del dominio applicativo (**business logic)** e si assicura che i dati siano affidabili prima di usarli per aggiornare la sorgente di informazione o per rispondere all'utente; le business rules regolano l'accesso dei client ai dati e l'elaborazione dei dati da parte dell'applicazione.

Modello MVC e Web Frameworks

L'architettura classica a 3-tier presenta un forte grado di accoppiamento tra i suoi moduli ed ogni modulo presenta difficoltà diverse a livello di sviluppo in quanto ricoprono campi differenti (si parla di MVC, Model View Controller); se si pensa che un web frameworks deve vincolare l'utente a seguire un tipo di architettura ne risulta che l'MVC (o più precisamente la Web-MVC in quanto tra model e view non c'è nessun legame per via del protocollo http che è stateless).
In questo contesto denotiamo:
Model: ha il compito del controllo e dell'accesso ai dati che deve passare al modulo Controller, la loro comunicazione è veicolata da un **ORM (Object Relational Mapping)** che definisce la relazione tra il modello dei dati concreti e quelli del modello ad oggetti dell'applicazione(?)
Controller: E' il modulo dove si trova tutta la logica vera e propria della nostra applicazione, gestisce le richieste HTTP, trasferisce dati alla View che riceve tramite comunicazione dal Model.
A tale livello abbiamo ad esempio meccanismi di sicurezza contro SQL-Injections,Clickjacking etc. , o meccanismi di URL Routing come l'**URL Routing RESTful** che è così chiamato perché segue le regole REST(REpresentational State Transfer:solo uso di CRUD operations, HTTP methods,URL parametrizzati con struttura gerarchica).
In generale, quindi, i Web frameworks offriranno supporto per tutte quelle che sono caratteristiche tipiche di un'applicazione web, inclusi per esempio il supporto al session-tracking e all'autenticazione dell'utente.I web frameworks, dunque, forniscono supporto per tutti (o alcuni de) i livelli del pattern Web-MVC, mettendo a disposizione dello sviluppatore uno stack di componenti software. Difatti,si parla di **full-stack frameworks** per riferirsi a quei frameworks che offrono supporto, attraverso componenti propriamente implementate o integrate da strumenti di terze parti, per ciascun livello dello stack software.
Possiamo poi definire anche i **micro-stack frameworks** che essenzialmente non forniscono supporto per l'intero stack per solo per uno. Ad esempio Flask è un esempio di micro-stack che gestisce il lato controller esclusivamente, quindi cose come http per esempio.Django è un esempio di full stack framework.

bisogna approfondire Django? boh

WebUML

UML (Unified Modeling Language) è un linguaggio di modellazione grafica e notazionale utilizzato per descrivere, progettare e comunicare sistemi software complessi. È ampiamente utilizzato nell'ingegneria del software per rappresentare visivamente la struttura, il comportamento, le interazioni e le relazioni dei componenti di un sistema. UML è un linguaggio standardizzato e supportato da molti strumenti di sviluppo software.

WebUML è un'estensione di UML specificamente progettata per modellare applicazioni web. Questa estensione è stata sviluppata per affrontare le particolari sfide associate allo sviluppo di applicazioni web interattive. WebUML fornisce una serie di diagrammi e notazioni specializzati che consentono di modellare aspetti specifici delle applicazioni web, come interazioni tra oggetti e pagine web, logiche lato client e server, form con submit, frameset e altre funzionalità web.

WebUML si basa sulla notazione di Conallen, che era uno dei primi approcci di modellazione delle interfacce web.

L'estensione WebUML offre diagrammi specifici per rappresentare le interazioni tra oggetti e pagine web in diversi contesti, ad esempio:

1. Server Page Diagram: Utilizzato per mostrare come i componenti del lato server interagiscono per generare e gestire le pagine web dinamiche.

2. Client Page Diagram: Utilizzato per rappresentare la struttura delle pagine web lato client e come i componenti interagiscono con l'utente attraverso il browser.

3. Link Diagram: Mostra le relazioni tra diverse pagine web attraverso i collegamenti ipertestuali.

4. Redirect Diagram: Rappresenta come le richieste di una pagina web vengono reindirizzate verso altre pagine.

WebUML è utile per progettisti e sviluppatori di applicazioni web perché consente di visualizzare e analizzare il flusso delle informazioni, le interazioni tra gli oggetti e le pagine web, nonché le logiche lato client e lato server. Ciò può facilitare la comprensione, la progettazione e il debugging di applicazioni web complesse.

JSP & Struts 2

Cos'è JSP:

JSP (JavaServer Pages) è una tecnologia utilizzata nello sviluppo di applicazioni web basate su Java. Consente di creare pagine web dinamiche mescolando codice Java e HTML all'interno di un unico file. Quando il server riceve una richiesta per una pagina JSP, questa viene compilata in una servlet Java, che poi viene eseguita per generare il contenuto HTML da inviare al browser del cliente.

Direttive:

Le direttive in JSP sono istruzioni che forniscono indicazioni al container JSP su come gestire la pagina. Le tre direttive principali sono:

1. `page`: Definisce le informazioni specifiche della pagina, come il linguaggio di scripting, l'encoding dei caratteri e altre impostazioni.
2. `include`: Permette di includere file esterni (ad esempio, un altro file JSP) all'interno della pagina corrente.
3. `taglib`: Utilizzata per importare tag personalizzati (taglib) definiti in librerie esterne.

Elementi di scripting:

Gli elementi di scripting in JSP consentono l'inserimento di codice Java all'interno delle pagine JSP. I tre principali elementi di scripting sono:

1. `<% %>`: Permette di inserire blocchi di codice Java.
2. `<%= %>`: Utilizzata per inserire il risultato di una singola espressione Java nel corpo della pagina.
3. `<%! %>`: Utilizzata per definire variabili e metodi globali nella classe JSP generata.

Oggetti Impliciti:

Gli oggetti impliciti sono oggetti predefiniti che JSP mette a disposizione all'interno del contesto delle pagine senza la necessità di essere dichiarati. Alcuni degli oggetti impliciti più comuni sono:

1. `**request**`: Fornisce accesso alle informazioni della richiesta HTTP.
2. `**response**`: Consente di manipolare la risposta HTTP da inviare al client.
3. `**session**`: Permette di memorizzare informazioni tra diverse richieste dello stesso client.
4. `**application**`: Rappresenta l'applicazione web e consente di condividere informazioni tra diverse sessioni e richieste.

Problemi di JSP:

Alcuni dei problemi associati all'uso di JSP includono:

1. Mescolanza di logica e presentazione: Poiché il codice Java è incorporato all'interno delle pagine JSP, c'è il rischio di creare applicazioni difficili da manutenere e comprendere a causa della mescolanza di logica e presentazione.

2. Sicurezza: L'uso non controllato di input utente nei JSP potrebbe portare a vulnerabilità come l'iniezione di script (XSS) e l'inclusione di file (Directory Traversal).

3. Performance: Poiché JSP viene tradotto in servlet Java, potrebbe esserci un piccolo overhead di prestazioni rispetto all'utilizzo diretto di servlet o framework di sviluppo.

4. Complessità: A mano a mano che le pagine JSP crescono di dimensione e complessità, la loro manutenzione può diventare problematica.

JavaBeans:

JavaBeans è un modello di programmazione di componenti riutilizzabili in Java. Un JavaBean è una classe Java che segue determinate convenzioni, come la presenza di un costruttore senza argomenti e metodi getter/setter per gli attributi. I JavaBeans sono comunemente utilizzati in combinazione con JSP per separare la logica dell'applicazione dalla presentazione.

Etichette Personalizzate:

Le etichette personalizzate, chiamate anche tag personalizzati, consentono agli sviluppatori di definire propri tag riutilizzabili, simili a quelli forniti dalle librerie standard di JSP. Questo permette di estendere la funzionalità di JSP in modo più modulare e organizzato.

Apache Struts 2:

Struts 2 è un framework per lo sviluppo di applicazioni web basate su Java. Fornisce un'architettura Model-View-Controller (MVC) per separare la logica dell'applicazione (Model), la presentazione (View) e il flusso di controllo (Controller). Struts 2 è il successore del framework Struts originale e presenta diverse migliorie, come una gestione più semplice delle azioni, un'interfaccia basata su oggetti per la gestione dei modelli, e una migliore gestione degli errori e delle eccezioni.

Utilizzando Struts 2, gli sviluppatori possono creare applicazioni web più organizzate, facili da mantenere e scalabili. Il framework offre anche supporto per la validazione dei dati, le chiamate AJAX e l'integrazione con diverse tecnologie web.

CSS

Cos'è il CSS:

Il CSS, acronimo di Cascading Style Sheets (Fogli di Stile a Cascata), è un linguaggio di programmazione utilizzato per definire l'aspetto e il layout degli elementi di un documento HTML o XML. Attraverso il CSS, è possibile controllare il colore, la dimensione, il tipo di carattere, la posizione e altri attributi dei contenuti presenti in una pagina web.

Le regole CSS:

Le regole CSS definiscono come gli elementi HTML devono essere visualizzati nel browser. Una regola è composta da un selettore e un blocco di dichiarazioni.

- Selettore: Indica quali elementi HTML devono essere selezionati e a cui applicare lo stile. Può essere un elemento HTML specifico, un gruppo di elementi o anche una classe o un'ID.

- Blocco di dichiarazioni: Contiene le proprietà CSS e i relativi valori che vengono applicati agli elementi selezionati dal selettore. Le proprietà possono essere ad esempio "color" (colore del testo), "font-size" (dimensione del carattere), "margin" (margine), "padding" (riempimento), ecc.

Modi di inserimento del CSS:

Ci sono tre principali modi di inserire il CSS in una pagina web:

1. CSS interno: Puoi includere il CSS all'interno del tag `<style>` all'interno dell'intestazione (`<head>`) del documento HTML. Il CSS interno si applica solo alla pagina specifica in cui è definito.

2. CSS esterno: Puoi creare un file esterno con estensione ".css" e collegarlo alla pagina HTML utilizzando l'elemento `<link>` all'interno dell'intestazione (`<head>`). Questo metodo consente di riutilizzare lo stesso foglio di stile su più pagine.

3. CSS inline: Puoi definire lo stile direttamente sugli elementi HTML utilizzando l'attributo `style`. Questo metodo è meno consigliato poiché rende il codice meno manutenibile e meno flessibile.

Sintassi dei selettori e classi:

I selettori CSS indicano quali elementi HTML devono essere selezionati per l'applicazione dello stile. Ci sono vari tipi di selettori:

- **Selettore di elemento:** Seleziona tutti gli elementi HTML corrispondenti al nome specificato, ad esempio p selezionerà tutti i paragrafi.

- **Selettore di classe:** Utilizza il simbolo del punto seguito da un nome per selezionare elementi che hanno assegnata la classe corrispondente. Ad esempio, `.classe` selezionerà tutti gli elementi con `class="classe"`

- **Selettore di ID**: Utilizza il simbolo cancelletto seguito da un nome per selezionare un elemento specifico con l'ID corrispondente. Ad esempio, `#mioID` selezionerà l'elemento con `id="mioID"`.

Conflitti e loro risoluzione:

I conflitti nel CSS possono verificarsi quando diverse regole si applicano allo stesso elemento. In caso di conflitto, il browser applicherà le regole basandosi su alcune regole di specificità. Ad esempio, una regola con un selettore di ID avrà maggiore priorità rispetto a una regola con un selettore di classe.

Se due regole hanno la stessa specificità, quella definita più tardi nel foglio di stile o nell'ordine di definizione avrà la priorità.

Fogli di stile dipendenti dai tipi di media:

Il CSS permette di specificare fogli di stile diversi da applicare a diverse tipologie di dispositivi o supporti di output, come schermi, stampanti o dispositivi mobili. Questo è possibile utilizzando le regole `@media` per i tipi di media specifici. Ad esempio:

```
/* Stile per schermi */
@media screen {
  body {
      font-size: 16px;
  }
}

/* Stile per stampa */
@media print {
  body {
      font-size: 12px;
  }
}
```

CSS con XML:

Il CSS può essere utilizzato anche con documenti XML. Tuttavia, poiché XML non ha una struttura fissa come HTML, è necessario specificare esplicitamente i selettori CSS in modo appropriato per adattarli ai tag e agli attributi specifici presenti nel documento XML.

Per utilizzare CSS con XML, è necessario associare uno spazio dei nomi CSS al documento XML e definire correttamente i selettori per gli elementi XML da stilizzare.

Esempio:

```xml
<?xml version="1.0" encoding="UTF-8"?>
<root xmlns="http://www.w3.org/1999/xhtml">
  <elemento>Contenuto</elemento>
</root>
```

```css
/* CSS per XML */
elemento {
  color: blue;
}
```

In questo esempio, lo stile sarà applicato all'elemento `<elemento>` all'interno del documento XML.

JAVASCRIPT

1. Cos'è JavaScript:
JavaScript è un linguaggio di programmazione ad alto livello, interpretato e basato sugli oggetti. È ampiamente utilizzato per aggiungere interattività e dinamicità alle pagine web. Viene spesso usato insieme ad HTML e CSS per creare siti web e applicazioni web interattive.

2. Dove va posto il suo codice e le sue proprietà:
Il codice JavaScript può essere inserito direttamente all'interno del corpo di un documento HTML tra i tag `<script>`. Per esempio:

```html
<!DOCTYPE html>
<html>
<head>
  <title>Titolo della pagina</title>
</head>
<body>
  <!-- Contenuto della pagina -->
```

```
  <script>
      // Codice JavaScript qui
  </script>
</body>
</html>
```

È anche possibile includere file JavaScript esterni utilizzando l'attributo `src` dell'elemento `<script>`:

```
<script src="percorso_del_file.js"></script>
```

3. Scope delle variabili:
In JavaScript, le variabili definite con `var` hanno uno scope di funzione o globale. Le variabili definite con `let` o `const` hanno uno scope di blocco (limitato all'interno delle parentesi graffe {} in cui sono dichiarate) o globale se dichiarate al di fuori di una funzione.

```
function esempioScope() {
  var variabileGlobale = 10; // Variabile globale all'interno della
funzione

  if (true) {
      let variabileLocale = 5; // Variabile locale al blocco if
      console.log(variabileGlobale); // Accessibile all'interno del
blocco
  }

  // console.log(variabileLocale); // Errore! La variabile non è
visibile qui
}

esempioScope();
```

4. Oggetti in JavaScript:
Gli oggetti sono fondamentali in JavaScript e sono delle collezioni di coppie chiave-valore, dove le chiavi sono le proprietà dell'oggetto e i valori sono i dati associati alle proprietà. Un oggetto può contenere sia dati che funzioni, queste ultime sono chiamate metodi dell'oggetto.

```
const persona = {
  nome: 'Mario',
  cognome: 'Rossi',
```

```
  età: 30,
  saluta: function() {
      console.log(`Ciao, sono ${this.nome} ${this.cognome}!`);
  }
};

console.log(persona.nome); // Output: "Mario"
persona.saluta(); // Output: "Ciao, sono Mario Rossi!"
```

5. Gli host di JavaScript:
JavaScript può essere eseguito in diversi ambienti o host, come i browser web, Node.js (che consente l'esecuzione di JavaScript lato server) e altre applicazioni che forniscono un motore JavaScript.

6. Variabili in JavaScript:
JavaScript è a tipizzazione dinamica, quindi le variabili possono contenere dati di diversi tipi durante l'esecuzione del programma. Le variabili possono essere dichiarate con `var`, `let` o `const`.

```
var x = 10; // Variabile con tipizzazione dinamica
let nome = 'Marco'; // Stringa
const PI = 3.14; // Costante
let isActive = true; // Booleano
let listaNumeri = [1, 2, 3]; // Array
```

7. Array associativi e multidimensionali:
Gli array in JavaScript sono oggetti speciali utilizzati per archiviare più valori in una singola variabile. Possono essere usati come array associativi, dove i valori sono accessibili attraverso una chiave.

```
const persona = {
  nome: 'Mario',
  cognome: 'Rossi',
  età: 30
};

console.log(persona['nome']); // Output: "Mario"
console.log(persona.età); // Output: 30
```

Gli array multidimensionali possono essere creati includendo array all'interno di array.

```
const matrix = [[1, 2], [3, 4], [5, 6]];
console.log(matrix[1][0]); // Output: 3
```

8. Operatori e funzioni:
JavaScript supporta una vasta gamma di operatori per eseguire operazioni su variabili, come operatori aritmetici (+, -, *, /), operatori di confronto (==, !=, ===, !==, >, <, >=, <=), operatori logici (&&, ||, !), etc.

Le funzioni sono blocchi di codice riutilizzabili che possono essere definiti e chiamati per eseguire operazioni specifiche.

```
function somma(a, b) {
  return a + b;
}

console.log(somma(2, 3)); // Output: 5
```

9. Tipi e Oggetti:
JavaScript è a tipizzazione dinamica, quindi le variabili possono cambiare tipo durante l'esecuzione del programma. Ci sono tipi primitivi come numeri, stringhe, booleani, null e undefined, e ci sono oggetti, che possono essere istanze di classi o oggetti personalizzati.

```
const numero = 10; // Tipo primitivo: numero
const testo = 'Ciao'; // Tipo primitivo: stringa
const isActive = true; // Tipo primitivo: booleano

const persona = {
  nome: 'Mario',
  età: 30
}; // Oggetto
```

10. Ereditarietà e prototipi:
In JavaScript, l'ereditarietà tra oggetti è basata su prototipi. Un oggetto può ereditare proprietà e metodi da un altro oggetto tramite il suo prototipo.

```
function Persona(nome) {
  this.nome = nome;
}
```

```
Persona.prototype.saluta = function() {
  console.log(`Ciao, sono ${this.nome}!`);
};

function Studente(nome, corso) {
  Persona.call(this, nome);
  this.corso = corso;
}

Studente.prototype = Object.create(Persona.prototype);
Studente.prototype.constructor = Studente;

Studente.prototype.presentati = function() {
  console.log(`Sono ${this.nome} e studio ${this.corso}.`);
};

const mario = new Studente('Mario', 'Matematica');
mario.saluta(); // Output: "Ciao, sono Mario!"
mario.presentati(); // Output: "Sono Mario e studio Matematica."
```

11. Incapsulamento:
JavaScript non supporta l'incapsulamento tradizionale come i linguaggi di programmazione orientati agli oggetti.

Tuttavia, è possibile simulare l'incapsulamento utilizzando chiusure (closures) per nascondere variabili e metodi all'esterno dell'oggetto.

12. HTML5 - Web Storage:
HTML5 introduce nuove funzionalità, tra cui Web Storage, che consente di memorizzare dati lato client. Le due tecnologie principali di Web Storage sono `localStorage` e `sessionStorage`, che consentono di archiviare dati chiave-valore persistenti e di sessione nel browser dell'utente.

```
// Salvare dati in localStorage
localStorage.setItem('nome', 'Mario');

// Recuperare dati da localStorage
const nome = localStorage.getItem('nome');

// Rimuovere dati da localStorage
localStorage.removeItem('nome');
```

DOM (Document Object Model)

Perché nasce:
Il DOM, acronimo di Document Object Model, nasce per fornire un'interfaccia di programmazione per i documenti HTML e XML. Prima dell'introduzione del DOM, la manipolazione e l'accesso agli elementi del documento HTML erano limitati e poco strutturati. Con l'implementazione del DOM, gli sviluppatori possono accedere agli elementi del documento, modificarli e aggiungere dinamicamente nuovi elementi utilizzando linguaggi di scripting come JavaScript.

Come è fatto:
Il DOM è una rappresentazione ad albero dei contenuti di un documento HTML o XML, dove ogni elemento del documento (tag, testo, attributi, ecc.) è rappresentato da un nodo nell'albero. I nodi del DOM sono organizzati gerarchicamente, partendo da un nodo radice che rappresenta l'intero documento e diramandosi in sotto-nodi che rappresentano gli elementi HTML/XML, i testi e gli attributi.

DOM W3C:
Il DOM è uno standard del World Wide Web Consortium (W3C). Il W3C definisce le specifiche del DOM per i diversi linguaggi di markup, come HTML e XML, in modo che gli sviluppatori possano interagire con i documenti web in modo uniforme e compatibile con i diversi browser.

Attributi:
Gli attributi sono informazioni aggiuntive associate agli elementi HTML o XML. Ad esempio, in un tag `<a>` (link), l'attributo "href" specifica l'URL di destinazione del link. I valori degli attributi possono essere letti, modificati o rimossi tramite il DOM.

Per esempio, con JavaScript è possibile accedere all'attributo "href" di un elemento `<a>` come segue:

```
// Accedere all'elemento <a>
const linkElement = document.querySelector('a');

// Leggere l'attributo "href"
const linkHref = linkElement.getAttribute('href');

// Modificare l'attributo "href"
linkElement.setAttribute('href', 'nuovo_url');

// Rimuovere l'attributo "href"
linkElement.removeAttribute('href');
```

Gestione degli eventi:
La gestione degli eventi nel DOM consente di catturare e gestire le interazioni dell'utente con gli elementi della pagina web. Gli eventi possono essere azioni come clic del mouse, pressione di tasti, passaggio del mouse su un elemento, caricamento della pagina, ecc.

Per gestire gli eventi, si utilizza il metodo `addEventListener` su un elemento HTML specifico, specificando l'evento da ascoltare e la funzione che deve essere eseguita quando l'evento si verifica.

Esempio:

```
<button id="myButton">Clicca qui</button>

<script>
  // Seleziona l'elemento button tramite l'ID
  const button = document.getElementById('myButton');

  // Aggiungi un listener per l'evento click
  button.addEventListener('click', function() {
      alert('Hai cliccato il pulsante!');
  });
</script>
```

In questo esempio, quando l'utente fa clic sul pulsante, verrà visualizzato un messaggio di avviso con la frase "Hai cliccato il pulsante!". La funzione anonima passata a `addEventListener` verrà eseguita quando l'evento "click" si verifica sull'elemento specificato.

AJAX (Asynchronous JavaScript and XML):

AJAX e SPA sono due concetti correlati utilizzati nello sviluppo web per migliorare l'esperienza dell'utente e l'efficienza delle applicazioni.
In generale Ajax permette di ottimizzare lo scambio di dati tra client e server andando a richiedere solo i dati necessari a costruire la parte della pagina che va modificata anziché ricaricare ogni volta tutta la pagina.

- **Perché nasce**: AJAX nasce per superare le limitazioni delle tradizionali applicazioni web, in cui la comunicazione tra il browser e il server richiedeva il ricaricamento completo della

pagina. Ciò limitava l'interattività e rallentava l'esperienza utente. Con AJAX, è possibile effettuare richieste asincrone al server e aggiornare solo determinate parti della pagina senza dover ricaricare l'intera pagina. Questo ha permesso di creare applicazioni web più fluide e responsive.

- **Precursori**: Prima dell'introduzione di AJAX, alcune tecnologie, come i frame e i plugin del browser, cercavano di migliorare l'interattività delle pagine web. Ma AJAX è stato il punto di svolta perché ha introdotto un approccio standardizzato e supportato da tutti i principali browser.

- **Ruolo dei diversi componenti**: Il modello AJAX si basa sugli strumenti seguenti:

JavaScript: È il linguaggio di scripting utilizzato per gestire le richieste al server,gli eventi, e l'aggiornamento delle pagine senza ricaricarle completamente .
DOM: per poter accedere e modificare la struttura della pagina HTML o XHTML;
XML: per rappresentare i dati che vengono scambiati tra server e client;
XMLHttpRequest (XHR): È un oggetto JavaScript utilizzato per effettuare richieste HTTP asincrone al server. È il cuore di AJAX e permette di inviare e ricevere dati senza interrompere l'interazione dell'utente con la pagina.
Server: Il server riceve le richieste del client (browser) attraverso XHR, elabora le richieste e invia i dati al client.

Ciclo di vita di XMLHttpRequest (XHR):

Considerati i seguenti passi:
1. A seguito di un evento generato dall'utente, che ad esempio preme un bottone, viene eseguita una chiamata JavaScript;
2. Viene creato e configurato un oggetto XMLHttpRequest; in particolare viene impostato un parametro di richiesta che include l'identificativo del componente che ha generato l'evento e qualsiasi valore immesso dall'utente;
3. L'oggetto XMLHttpRequest manda al server una richiesta HTTP in modo asincrono rispetto all'azione dell'utente. Il server web elabora la richiesta come farebbe normalmente (ad esempio, attraverso una servlet) e, se necessario, salva i dati passati nella richiesta in modo persistente;
4. Il server web prepara quindi una risposta HTTP nel cui body viene inserito un documento XML con tutti gli aggiornamenti che sono stati richiesti dal client;
5. L'oggetto XMLHttpRequest riceve i dati XML, li elabora e aggiorna opportunamente la DOM della pagina (X)HTML visualizzata dal browser

SPA (Single Page Application):
- **Perché nasce**: Le Single Page Application (SPA) nascono per offrire un'esperienza di navigazione più fluida e veloce. In una SPA, l'intera applicazione web viene caricata una sola volta al momento del primo accesso, e successivamente solo i contenuti necessari vengono scaricati in background e visualizzati dinamicamente senza ricaricare completamente la pagina.

- **Precursori:** Le prime idee delle SPA possono essere fatte risalire agli anni 2000, ma il concetto è diventato più popolare con l'aumento dell'utilizzo di AJAX, che ha consentito il caricamento dinamico dei contenuti senza dover ricaricare la pagina.

- **Ruolo dei diversi componenti:** In una SPA, l'applicazione web è tipicamente realizzata utilizzando un framework frontend come Angular, React o Vue.js. Questo framework si occupa di gestire il routing, lo stato dell'applicazione e il rendering dei contenuti dinamici. Gli utenti interagiscono principalmente con l'applicazione tramite il browser e il framework frontend gestisce la comunicazione con il server tramite richieste AJAX per ottenere dati aggiornati o inviare dati al server.

Vantaggi:
• non occorre ricaricare sempre la pagina completa, con un miglioramento delle prestazioni e della dinamicità dell'applicazione
Svantaggi:
• rende più difficile l'indicizzazione del sito da parte dei motori di ricerca
• problemi nella navigazione (il back va gestito dal browser)
• problemi nell'analisi del funzionamento del sito
• in generale, è più difficile da gestire

In sintesi, AJAX e SPA hanno contribuito significativamente a migliorare l'esperienza utente nel web moderno. AJAX ha permesso richieste asincrone al server, evitando ricaricamenti completi delle pagine, mentre le SPA hanno spinto l'idea di applicazioni web che funzionano come una singola pagina, offrendo un'esperienza di navigazione più fluida e reattiva.

Semantic Web & RDF

Il Web semantico è una visione dell'evoluzione del World Wide Web in cui i dati hanno un significato ben definito e strutturato, consentendo ai computer di comprenderli e processarli in modo più intelligente. L'obiettivo è andare oltre la semplice visualizzazione di informazioni per rendere il web una fonte di conoscenza e intelligenza artificiale.

• Linked Data (Dati Collegati): È una pratica per la pubblicazione e l'interconnessione dei dati su Internet in modo che siano facilmente accessibili e collegabili tra loro. Ciò viene realizzato utilizzando standard aperti e protocolli web, come HTTP e URI, per identificare e recuperare i dati. L'idea chiave è quella di consentire ai dati di essere collegati tra diverse fonti, consentendo così di ottenere un panorama più completo e approfondito di informazioni.

• Open Data (Dati Aperti): Sono dati che possono essere liberamente utilizzati, ripubblicati e redistribuiti da chiunque, senza restrizioni di copyright, brevetti o altre limitazioni legali. Questo promuove la trasparenza, la collaborazione e l'innovazione, poiché le persone possono accedere a queste risorse e utilizzarle in modi creativi.

• Linked Open Data (Dati Aperti Collegati): È una combinazione di Linked Data e Open Data. I dati aperti collegati sono dati aperti che seguono principi di dati collegati, consentendo quindi di collegarli ad altre fonti di dati e di creare un vasto ecosistema di dati interconnessi.

• Rappresentazione tramite Grafo: Nella Web semantico, i dati vengono spesso rappresentati utilizzando un modello a grafo. Un grafo è un insieme di nodi (entità) collegati da archi (relazioni). Nella rappresentazione dei dati tramite grafo, le entità sono solitamente identificate utilizzando URI (Uniform Resource Identifier) e le relazioni tra di esse vengono definite in modo strutturato utilizzando il Resource Description Framework (RDF).

• RDF (Resource Description Framework): È uno dei principali standard utilizzati nel Web semantico per rappresentare dati strutturati. Con RDF, è possibile esprimere le informazioni in forma di triplette (soggetto, predicato, oggetto), dove il soggetto e l'oggetto sono le entità, e il predicato rappresenta la relazione tra di esse. Ad esempio, "Maria" (soggetto) "è amica di" (predicato) "Luca" (oggetto). Utilizzando queste triplette RDF, è possibile creare grafi di conoscenza interconnessi e facilitare la condivisione e la comprensione delle informazioni da parte delle macchine.

Microservizi

Architettura Monolitica:

Cos'è: L'architettura monolitica è un approccio tradizionale per la progettazione di applicazioni software, in cui tutti i componenti funzionali dell'applicazione sono combinati in un singolo pacchetto (il monolite). Questo significa che l'intera applicazione viene sviluppata, implementata e distribuita come un'unica unità.

Vantaggi:
1. Semplice da sviluppare: Poiché tutti i componenti sono all'interno dello stesso pacchetto, è più facile sviluppare e testare l'applicazione.
2. Meno complessità iniziale: L'avvio di un progetto monolitico richiede meno configurazione e infrastruttura rispetto a un'architettura a microservizi.
3. Prestazioni: In alcuni casi, l'architettura monolitica può essere più efficiente in termini di prestazioni, poiché non c'è overhead di comunicazione tra servizi.

Svantaggi:
1. Scalabilità: L'intera applicazione viene scalata, il che può comportare sprechi di risorse quando solo alcune parti dell'applicazione richiedono scalabilità.
2. Dipendenza tecnologica: Poiché tutti i componenti sono vincolati alla stessa tecnologia, l'aggiornamento di uno di essi può comportare l'aggiornamento di tutto il sistema.
3. Complessità gestionale: Man mano che l'applicazione cresce, diventa sempre più difficile gestire e mantenere il codice.

Architettura a Microservizi:

Cos'è: L'architettura a microservizi è un approccio moderno per progettare applicazioni software, in cui l'applicazione è suddivisa in piccoli servizi indipendenti che collaborano tra

loro per fornire funzionalità specifiche. Ogni microservizio rappresenta una parte del dominio funzionale dell'applicazione.

Motivo della creazione: L'architettura a microservizi è stata sviluppata per affrontare i problemi di complessità e scalabilità delle architetture monolitiche. Essa cerca di affrontare questi problemi suddividendo l'applicazione in unità più piccole e gestibili.

Gestione della complessità: L'approccio a microservizi affronta la complessità suddividendo l'applicazione in servizi autonomi, ciascuno con uno scopo ben definito. Ciò semplifica lo sviluppo, la manutenzione e la scalabilità dell'applicazione.

Quando utilizzarla: L'architettura a microservizi è adatta a progetti complessi o di grandi dimensioni, dove diversi team possono lavorare indipendentemente su diversi servizi. È particolarmente utile quando è necessaria una maggiore scalabilità, resilienza e agilità nello sviluppo.

I suoi punti chiave:
1. Organizzazione: Suddivide l'applicazione in servizi autonomi, ognuno responsabile di una funzionalità specifica.
2**. Design**: Favorisce un design di servizi indipendenti con interfacce chiare e ben definite.
3. Comunicazione: I microservizi comunicano tra loro tramite API, spesso attraverso protocolli RESTful o message broker.
4. Gestione dati: Ogni servizio può avere il proprio database o condividere database con altri servizi, a seconda delle esigenze.

Differenze tra le due architetture:

1. Dimensione e complessità: L'architettura monolitica è un'entità unica, mentre l'architettura a microservizi è suddivisa in servizi più piccoli e gestibili.
2. Scalabilità: L'architettura a microservizi permette una maggiore scalabilità, poiché solo i servizi che richiedono l'aggiornamento possono essere scalati indipendentemente.
3. Dipendenze: Nell'architettura monolitica, i componenti sono fortemente vincolati alla stessa tecnologia, mentre nell'architettura a microservizi, i servizi possono essere sviluppati e distribuiti indipendentemente.
4. Gestione e manutenzione: L'architettura monolitica può essere più semplice da gestire e mantenere in fase iniziale, ma diventa più complessa man mano che l'applicazione cresce. L'architettura a microservizi richiede uno sforzo iniziale maggiore, ma può semplificare la gestione a lungo termine grazie alla suddivisione dei servizi.
5. Comunicazione tra componenti: Nell'architettura monolitica, la comunicazione tra componenti avviene all'interno del monolite, mentre nell'architettura a microservizi, la comunicazione avviene spesso tramite API e richieste RESTful o altri meccanismi di messaggistica.

La programmazione incontra il web

Il mondo della programmazione ha incontrato il web in vari modi, tra cui l'utilizzo di bot, crawler, scrapers e web driver. Vediamo in dettaglio ciascuno di questi aspetti:

Bot Internet:
Un bot Internet è un programma software che automatizza compiti specifici su Internet. I bot possono essere utilizzati per una vasta gamma di scopi, tra cui indicizzazione dei motori di ricerca, risposte automatiche a domande frequenti, interazione con i social media e molto altro. I bot possono essere utilizzati in modo positivo per migliorare l'efficienza e l'esperienza utente, ma possono anche essere abusati per scopi dannosi, come il cosiddetto "web scraping" non etico.

Bot exclusion Standard:
Il bot exclusion standard, comunemente noto come "robots.txt", è un file di testo che viene posizionato nella radice di un sito web per comunicare ai bot quali pagine o sezioni del sito possono essere esplorate e quali devono essere evitate. È uno strumento utilizzato dagli amministratori di siti web per controllare quali parti del loro sito possono essere indicizzate dai motori di ricerca e quali no.

Proprietà di un buon bot:
Un buon bot Internet dovrebbe essere etico, rispettare le regole imposte dagli amministratori del sito web (se presenti nel robots.txt), essere efficiente nel compiere i compiti assegnati, fornire risposte precise e rilevanti quando è coinvolto in interazioni con gli utenti e non sovraccaricare il server del sito web con richieste eccessive.

Programmazione di un Bot Internet:
La programmazione di un bot Internet coinvolge lo sviluppo di script o programmi che possono automatizzare determinate azioni sul web. Questo può essere fatto utilizzando linguaggi di programmazione come Python, JavaScript, PHP, ecc. Tuttavia, è importante notare che l'automazione su alcuni siti potrebbe violare i loro termini di servizio, quindi è sempre essenziale assicurarsi di rispettare le politiche e le regole del sito.

Crawler:
Un crawler (noto anche come spider o web crawler) è un tipo di bot che analizza il web in modo sistematico per raccogliere informazioni e indicizzare i contenuti dei siti web. I motori di ricerca utilizzano crawler per esplorare il web e costruire i loro database di pagine web.

Web Crawling:
Il web crawling è il processo attraverso il quale un crawler naviga in modo automatico attraverso il web, visitando pagine web e seguendo i collegamenti per scoprire nuove pagine da indicizzare.

Architettura di un Crawler:
Un crawler è generalmente costituito da un insieme di componenti, tra cui un modulo per scaricare il contenuto delle pagine web, un modulo per analizzare il contenuto scaricato, un database per archiviare le informazioni raccolte e un algoritmo di scoperta dei collegamenti per trovare nuove pagine.

Framework per la creazione dei Crawler:
Esistono diversi framework disponibili per aiutare nella creazione dei crawler, tra cui Scrapy in Python e Apache Nutch in Java.

Sottile differenza tra Crawler e Spider:

Nel contesto del web e dell'indicizzazione dei contenuti, i termini "crawler" e "spider" sono spesso usati in modo intercambiabile, ma possono avere delle sottili differenze di significato. Tuttavia, è importante notare che nel linguaggio comune e nella pratica quotidiana, le due parole sono usate in modo intercambiabile e spesso si riferiscono alla stessa cosa: un programma o un bot che esplora il web per raccogliere informazioni.

Ecco una spiegazione delle possibili differenze:

1. **Crawler**:
Il termine "crawler" è generalmente usato per indicare un programma o uno script che viene utilizzato per esplorare sistematicamente il web visitando pagine web e recuperando dati da queste pagine. I crawler sono spesso utilizzati dai motori di ricerca per indicizzare le pagine web e costruire i loro database. L'obiettivo principale di un crawler è quello di esplorare il web il più ampiamente possibile e raccogliere informazioni.

2. **Spider**:
Il termine "spider" è spesso utilizzato in modo più specifico per riferirsi a un tipo di crawler che ha la capacità di seguire i collegamenti delle pagine web (come un ragno che segue i fili di una tela), visitando una pagina e poi passando a quelle collegate da essa. I spider sono in grado di "seguire" i link per scoprire nuove pagine e continuare l'esplorazione nel web in modo più organizzato.

In sostanza, si potrebbe dire che tutti gli spider sono crawler, ma non tutti i crawler sono necessariamente spider. Un crawler generico può limitarsi a esplorare pagine web senza seguire attivamente i collegamenti, mentre uno spider si concentra sull'esplorazione "a rete" seguendo i collegamenti da una pagina all'altra.

Scrapers:
Lo scraping web è il processo di estrazione di dati strutturati da pagine web. I scrapers sono utilizzati per estrarre informazioni specifiche da siti web e trasformarle in un formato utilizzabile, ad esempio un file CSV o un database.

Limiti dell'approccio tradizionale:
L'approccio tradizionale allo scraping web è limitato quando si tratta di interagire con pagine altamente dinamiche o pagine web che utilizzano tecniche avanzate come JavaScript per caricare i contenuti in modo asincrono. In tali casi, uno scraper tradizionale potrebbe non essere in grado di ottenere i dati desiderati.

Interazione con pagine web altamente dinamiche:
Per affrontare il problema delle pagine web altamente dinamiche, è possibile utilizzare web driver come Selenium.

Web driver - Selenium:
Selenium è un framework di automazione del browser che consente di controllare un browser in modo programmatico. Questo consente di simulare l'interazione umana con le pagine web e di gestire pagine altamente dinamiche che richiedono l'esecuzione di JavaScript. Selenium supporta diversi linguaggi di programmazione come Python, Java, C#, ecc.
Selenium migliora l'esperienza di web crawling e web scraping su pagine web dinamiche rispetto all'approccio tradizionale in diversi modi:

1. Interazione con pagine web dinamiche: Le pagine web dinamiche utilizzano tecniche avanzate come JavaScript per aggiornare il contenuto in modo asincrono senza dover ricaricare completamente la pagina. L'approccio tradizionale al web scraping, basato su richieste HTTP dirette, non è in grado di gestire efficacemente queste dinamiche. Selenium, invece, utilizza un vero e proprio browser (come Google Chrome o Mozilla Firefox) tramite il web driver, consentendo di eseguire JavaScript e di interagire con elementi dinamici come pulsanti, form e altri elementi interattivi.

2. Rendering completo della pagina: I motori di ricerca e gli approcci tradizionali di scraping possono ottenere solo l'HTML grezzo della pagina. Tuttavia, alcune informazioni cruciali potrebbero essere generate tramite JavaScript e quindi mancare nel codice sorgente dell'HTML. Selenium, grazie al rendering completo della pagina web tramite un vero browser, può accedere al DOM (Document Object Model) completamente modificato e aggiornato, consentendo di estrarre dati aggiornati e completi da pagine web dinamiche.

3. Gestione delle sessioni e dell'autenticazione: Molti siti web richiedono autenticazione o interazioni complesse per accedere ai dati desiderati. Selenium consente di gestire queste interazioni, come l'inserimento delle credenziali di accesso o la navigazione attraverso pagine successive di un sito per raggiungere il contenuto desiderato. Con l'approccio tradizionale, gestire queste sessioni complesse sarebbe più complicato.

4. Debugging e testing: Selenium è ampiamente utilizzato per il web testing, consentendo agli sviluppatori di automatizzare i test e verificare che l'interfaccia utente funzioni correttamente. Questo aspetto è particolarmente utile quando si esegue lo scraping da siti web complessi, in quanto Selenium può essere utilizzato per testare il codice di scraping e risolvere eventuali problemi o errori di interazione con la pagina.

5. Supporto multi-browser: Selenium supporta diversi browser, tra cui Google Chrome, Mozilla Firefox, Microsoft Edge e altri. Questo offre una maggiore flessibilità nel scegliere il browser più adatto alle proprie esigenze di scraping o crawling.

Selenium migliora l'esperienza di web crawling e web scraping su pagine web dinamiche rispetto all'approccio tradizionale consentendo l'interazione con pagine dinamiche, il rendering completo della pagina, la gestione delle sessioni e dell'autenticazione e offrendo un supporto multi-browser. Questi vantaggi rendono Selenium uno strumento potente e flessibile per l'estrazione di dati da siti web complessi e dinamici, fornendo risultati accurati ed aggiornati. Tuttavia, è sempre importante utilizzare Selenium in modo etico, rispettando le politiche dei siti web e senza abusarne per attività di scraping non autorizzato.

In sintesi, la programmazione incontra il web attraverso l'automazione con bot Internet, l'esplorazione con crawler, l'estrazione di dati con scrapers e l'interazione con pagine dinamiche tramite web driver come Selenium. Tuttavia, quando si utilizzano queste tecniche, è essenziale essere etici, rispettare le regole dei siti web e assicurarsi di non violare i termini di servizio.

End-to-End Testing relativo alle Tecnologie Web:

A cosa serve:
L'End-to-End Testing è un tipo di testing che verifica il corretto funzionamento di un'applicazione o un sistema web dall'inizio alla fine, simulando il flusso delle azioni che gli utenti effettuano durante l'utilizzo. Questo tipo di testing è essenziale per rilevare eventuali problemi che possono verificarsi nell'interazione tra diversi componenti del sistema.

I suoi passaggi chiave:
Il processo di End-to-End Testing può variare in base alle specifiche dell'applicazione o del sistema in questione, ma in generale, comprende i seguenti passaggi chiave:
a. Identificazione degli scenari di test: Definire i casi di test che coprono le principali funzionalità dell'applicazione.
b. Configurazione dell'ambiente di test: Preparare l'ambiente in cui verranno eseguiti i test, che può essere simile o identico a quello di produzione.
c. Esecuzione dei test: Condurre i test seguendo gli scenari identificati, simulando le azioni degli utenti.
d. Verifica dei risultati: Confrontare i risultati ottenuti durante i test con i risultati attesi per identificare eventuali discrepanze.
e. Risoluzione dei problemi: Segnalare e correggere eventuali errori o bug trovati durante i test.

Tipi di approccio o Manuale / Automatico:
L'End-to-End Testing può essere eseguito manualmente o automaticamente.

- **Approccio Manuale**: Nel testing manuale, gli operatori eseguono i test interagendo con l'applicazione come farebbe un utente reale. Questo metodo può essere soggetto a errori umani e richiede una maggiore quantità di tempo e risorse.

- **Approccio Automatico:** Nel testing automatico, vengono utilizzati strumenti e script per eseguire i test senza intervento umano diretto. Questo approccio è più veloce, efficiente e ripetibile.

Selezione degli elementi:
Durante l'End-to-End Testing, è necessario selezionare gli elementi dell'interfaccia utente che verranno sottoposti a test. Ciò può essere fatto attraverso diversi tipi di selezionatori:

- **Coordinate assolute dello schermo**: Identificare gli elementi tramite le loro coordinate esatte sullo schermo.
- **Selettori basati su Layout**: Utilizzare selettori basati sulla struttura HTML o XML dell'applicazione.
- **Selettori visivi**: Rilevare gli elementi tramite caratteristiche visive, come colore, forma o dimensione.

5. Come ottenere test:
Per eseguire l'End-to-End Testing, è possibile utilizzare diverse tecniche:

- **Programmable Testing**: Scrivere script di test utilizzando linguaggi di scripting come Python o JavaScript per automatizzare le azioni degli utenti.
- **Capture & Replay**: Registrare una serie di azioni manuali eseguite dall'utente e riprodurle successivamente in modo automatico.
- **Fully Automated Test Generatio**n: Utilizzare strumenti che generano automaticamente casi di test in base all'analisi dell'applicazione.

6. **Problematiche rilevanti nel Testing**:
Durante l'End-to-End Testing, possono sorgere diverse problematiche:

- **Instabilità**: A causa di cambiamenti nell'ambiente o nell'applicazione, i test possono diventare instabili e produrre risultati inconsistenti.

• Dipende tipicamente da fattori esterni
• La pagina stando caricando più lentamente non ha ancora caricato elementi alla quale il test cerca di accedere
• Soluzione: Usare time-out adeguati e aspettare che gli elementi sui quali bisogna agire siano caricati

- **Fragilità**:
La fragilità nei test si riferisce alla tendenza dei test a diventare instabili e a fallire in modo errato a causa di piccoli cambiamenti nell'interfaccia utente o nell'ambiente. Questo può accadere quando i test dipendono troppo da selezionatori rigidi o da elementi specifici dell'interfaccia utente che cambiano frequentemente. La fragilità dei test può portare a una gestione complicata dei test e richiede costanti aggiornamenti dei casi di test per mantenere la loro affidabilità.

▪ **Soluzione (Localizzatori**):
Per risolvere la fragilità dei test, una soluzione comune è utilizzare localizzatori dinamici e resilienti. Invece di selezionare gli elementi tramite coordinate fisse o selettori statici, i localizzatori si adattano in modo intelligente alle modifiche dell'interfaccia utente. Ciò può essere ottenuto utilizzando attributi unici o pattern stabili per identificare gli elementi, in modo che i test siano meno sensibili a piccole variazioni nella struttura dell'interfaccia utente. L'uso di localizzatori più robusti può aumentare la stabilità dei test e ridurre la necessità di aggiornamenti frequenti.

▪ **Problema dell'evoluzione del test**:
Il problema dell'evoluzione del test si verifica quando l'applicazione viene aggiornata o modificata nel tempo, rendendo obsoleti alcuni casi di test esistenti. Questo può portare a test falliti o test non più rappresentativi delle funzionalità dell'applicazione, poiché il sistema è cambiato ma i test non sono stati adeguati di conseguenza.

• **Soluzione (Object Page**):
Per affrontare il problema dell'evoluzione del test, un'approccio utile è utilizzare il pattern Object Page. Questo pattern organizza gli elementi dell'interfaccia utente e le azioni associate in oggetti logici, separando così la logica di test dalla struttura dell'applicazione. Quando avviene un cambiamento nell'applicazione, sarà necessario apportare modifiche solo agli oggetti logici, piuttosto che dover modificare ciascun caso di test individualmente. Ciò semplifica la manutenzione dei test e li rende più flessibili all'evoluzione dell'applicazione. Utilizzando Object Page, è possibile isolare il codice del test dagli elementi dell'interfaccia utente, facilitando così il processo di aggiornamento dei test quando cambiano le funzionalità dell'applicazione.

Domande:

HTTP

Cos'è HTTP?

HTTP è un protocollo di comunicazione di rete tra entità remote, esso è un protocollo stateless ossia senza stato e dunque non mantiene nessuno stato della comunicazione;per farlo occorre implementare opportuni meccanismi.
Esso si compone di un meccanismo di richieste(spesso effettuate da un client) e risposte (spesso effettuate da un server).
La richiesta è composta da:

1. Request Line (composta così: [TipoRichiesta(GET-POST.etc)] [risorsa alla quale si effettua la richiesta] [HTTP VERSION]
2. Vari HEADER (host,tipo accettato,auth header,user agent,campo if-modified per i cookie etc)
3. linea vuota
4. BODY (che può essere anche vuoto)

La risposta è composta da:

1. STATUS LINE (composta così: [VERSIONE] [STATUS CODE(200,300,400,etc)+PHRASE
2. Vari HEADER (host,tipo accettato,auth header,user agent,campo if-modified per i cookie etc)
3. linea vuota
4. BODY (che può essere anche vuoto)

Messaggi HTTP ;quali sono le informazioni contenute in un cookie? e cos'è un cookie?

Un cookie è un pezzo di informazione testuale, che serve per vari scopi e può contenere vari tipi di informazione.
Esso viene sempre creato dal server e passato al client; spesso il cookie ha una data di scadenza segnata dopo la quale viene eliminato ed inoltre può essere anche eliminato dal server mandando un cookie uguale ma con data di scadenza superata per far avvenire la cancellazione.
Esso contiene informazioni quali: nome del cookie, informazione specifica del cookie,dominio e path associati,e data di scadenza.
I suoi usi più comuni sono nel session tracking e nell'autenticazione.

Come può il server cancellare un cookie?

Se il server vuole che un cookie venga cancellato, ne invia quindi uno con lo stesso nome, host e path di quello da cancellare, ma con expiration date negativa, in modo che quest'ultimo venga prima sovrascritto e immediatamente cancellato.

Si spieghi come HTTP offre un meccanismo di caching

HTTP (Hypertext Transfer Protocol) offre un meccanismo di caching che consente di memorizzare temporaneamente le risposte alle richieste dei client per accelerare le future richieste simili. Il caching in HTTP è fondamentale per ottimizzare le prestazioni del web e ridurre il carico sui server web.
Le risorse HTTP come ad esempio le pagine web possono essere contrassegnate come cacheable, e quindi salvate in cache dal client.
E' presente anche un campo max-age che ci dice per quanto tempo può essere memorizzata in cache la risorsa.
Tramite un campo If-Modified il server può capire se rinviare la pagina o meno, in caso in cui la pagina non è stata modificata rispetto alla data indicata il server risponderà con un 304 Not Modified e non invierà di nuovo la pagina al client.

Si spieghi come HTTP offre un meccanismo di autenticazione

Quando un utente tenta di accedere per la prima volta ad una risorse che necessita di autorizzazione,il server risponderà con “401 Unauthorized” e dopodichè chiederà le credenziali, tramite un apposita form,e attraverso un campo dell’header Authorization il client le invierà in base64 al server che ne effettuerà la verifica e in caso positivo le utilizzerà nelle richieste successive.

Come si presenta una richiesta HTTP?
La richiesta è composta da:

1. Request Line (composta così: [TipoRichiesta(GET-POST.etc)] [risorsa alla quale si effettua la richiesta] [HTTP VERSION]
2. Vari HEADER (host,tipo accettato,auth header,user agent,campo if-modified per i cookie etc)
3. linea vuota
4. BODY (che può essere anche vuoto)

HTML

Cos' è HTML?
L’Hypertext Markup Language è un linguaggio di markup utilizzato per la creazione di pagine web, esso utilizza i gli elementi, detti anche <tag>, per definire regole precise di renderizzazione di quest’ultimi agli user-agent che usualmente sono client web; esso ha una struttura ad albero.
Ad oggi si è arrivati alla versione HTML5.

Quali sono le principali innovazioni introdotte da HTML5?
Ad oggi si è arrivati alla versione HTML5, esso rimuove in primis tag obsoleti e si adatta a supportare al meglio strumenti quali CSS per il rendering dello stile e javascript per lo scripting al quale delega quasi completamente questi compiti oramai; introduce supporto nativo ad audio e video, depreca le applet,introduce supporto alle API di geolocalizzazione e web storage, canvas per la grafica vettoriale, supporto al drag & drop all’interno delle pagine.r

Come funziona il web storage in HTML5? Ha a che fare con il session tracking? In che senso?

Il Web Storage in HTML5 è un meccanismo che consente di memorizzare dati sul lato client (nella macchina dell'utente) in modo persistente tra le sessioni di navigazione, ma non ha direttamente a che fare con il session tracking.

Il session tracking è una tecnica utilizzata per tenere traccia delle sessioni degli utenti in un'applicazione web. Di solito, viene implementato attraverso l'uso di cookie o URL con parametri di sessione. L'obiettivo è mantenere lo stato dell'utente durante una sessione di navigazione, ad esempio, per gestire l'autenticazione o conservare le informazioni dello shopping cart.

Il Web Storage in HTML5 non è direttamente correlato al session tracking, poiché non è progettato per tenere traccia delle sessioni utente. Tuttavia, il Web Storage può essere utilizzato in combinazione con il session tracking o altri meccanismi per memorizzare dati specifici della sessione o informazioni di autenticazione in modo più efficiente rispetto ai cookie tradizionali. Ad esempio, è possibile utilizzare il `sessionStorage` per memorizzare temporaneamente dati di sessione, ma è necessario implementare ulteriori meccanismi per la gestione delle sessioni e per l'identificazione dell'utente, come l'utilizzo di token di sessione o di autenticazione.

Quali sono le differenze tra l'attributo NAME e l'attributo ID in HTML?

L'attributo id definisce univocamente un elemento HTML nella pagina,quindi non può essere creato un altro elemento con lo stesso id.

L'attributo name non definisce univocamente un elemento HTML , viene utilizzato nei campi di un form per recuperare il nome del campo e il suo valore quando viene effettuato il submit.

Differenze tra HTML e CSS

La differenza principale tra HTML e CSS è che il primo definisce gli elementi che fanno parte di una pagina HTML, CSS ne definisce invece lo stile.

CSS

Cos'è CSS?

Cascade Style Sheet CSS è un linguaggio utilizzato per definire la formattazione e lo stile di pagine in linguaggi quali HTML,XML,XHTML etc.

Esso può essenzialmente definire cose come colori,allineamenti,font, per ogni elemento della pagina tramite appositi *"selettori"*

Che sintassi possono seguire i selettori in CSS?

elemento: {proprietà: attributo}

h1 :{color: blue}

Regole CSS: in CSS posso imporre che tutti gli item di una lista numerata vengano scritti in rosso e tutti quelli di una lista non numerata in verde? in che modo?

```
/* Lista non numerata */
ul {
  color: green;
}

/* Lista numerata */
ol {
  color: red;
}
```

in CSS posso imporre che il primo elemento di una lista venga scritto in rosso e tutti gli altri in verde?

```
/* Il primo elemento della lista sarà rosso */
li:first-child {
  color: red;
}

/* Tutti gli altri elementi della lista saranno verdi */
li:not(:first-child) {
  color: green;
}
```

Nella realizzazione di un documento HTML vogliamo che il primo elemento di ogni lista,sia essa ordinata o meno, sia rappresentato in rosso, e tutti gli altri in blu. Si proponga una soluzione.

```
/* Il primo elemento di ogni lista sarà rosso */
ul li:first-child,
ol li:first-child {
  color: red;
```

```
}

/* Tutti gli altri elementi di ogni lista saranno blu */
ul li:not(:first-child),
ol li:not(:first-child) {
  color: blue;
}
```

Si scriva una riga CSS che stampi in grassetto tutti gli elementi di una lista ordinata che è parte di un'altra lista a livello superiore.

Si può ottenere ciò utilizzando il selettore CSS `ol ol li`, che selezionerà tutti gli elementi di una lista ordinata che è parte di un'altra lista ordinata a livello superiore, e quindi puoi applicare il grassetto con la proprietà `font-weight`.

```
ol ol li {
  font-weight: bold;
}
```

differenze tra XSL e CSS (CSS e XSL)

XSL (eXtensible Stylesheet Language) e CSS (Cascading Style Sheets) sono due tecnologie utilizzate per la formattazione e la presentazione di documenti in un ambiente web, ma hanno scopi e funzionalità diverse. Ecco le principali differenze tra XSL e CSS:

Il confronto di base non sussiste in quanto sono elementi utilizzati per scopi diversi,hanno sintassi completamente diversa e l'utilizzo lato client di xsl è poco comune rispetto al css

Differenza Principale:

- CSS: È principalmente utilizzato per controllare l'aspetto e la formattazione di documenti HTML o XML. Si concentra sulla presentazione visiva del contenuto web, come il colore del testo, il layout delle pagine e il posizionamento degli elementi.
- XSL: È una famiglia di linguaggi di stile utilizzati per la trasformazione di documenti XML in altri formati, come HTML, XHTML, PDF o testo. La sua funzione principale è la trasformazione e la manipolazione dei dati XML.Ci sono linguaggi basati su XSL come XSLT.

Javascript

Cos'è? differenze con Java?esistono i tipi di valore in JavaScript? se si, quali sono?

JavaScript è un linguaggio di scripting basato su oggetti, nonostante abbia java nel nome ha poco in comune con quest'ultimo.Le differenze principali sono:

Java	Javascript
Usato per scopi diversi	Solo per programmazione Web
Compilato	Non compilato
Codice compilato in applets separati dal sorgente (ad es. HTML)	Codice direttamente incluso nel sorgente (ad es. HTML)
Linguaggio di programmazione	Linguaggio di scripting
Controllo dei tipi delle variabili forte	Controllo dei tipi delle variabili debole
Controllo di accesso a più livelli (private, protected, public)	Controllo di accesso semplice
Rigida verifica degli array	Nessuna verifica degli array
Gerarchia delle classi	Gerarchia delle istanze

Tabella 12.1: Principali differenze tra Java e Javascript

La tabella è presa dagli appunti della Corazza, tuttavia va aggiunto che javascript è interpretato e che ad oggi è usabile anche per lo sviluppo back-end e altri molteplici scopi tramite specifici framework (node.js ad esempio).

JavaScript è un linguaggio di scripting object based, utilizzato per la programmazione lato client

tipi di valore in JavaScript?

I tipi di dato previsti da JavaScript sono 8 (7 primitivi + Object).

Essi sono:

- Boolean
- Null
- Undefined
- Number
- BigInt
- String
- Symbol

- Object

scope delle variabili in JS (*)?

In JavaScript, le variabili definite con `var` hanno uno scope di funzione o globale. Le variabili definite con `let` o `const` hanno uno scope di blocco (limitato all'interno delle parentesi graffe {} in cui sono dichiarate) o globale se dichiarate al di fuori di una funzione.

```
function esempioScope() {
  var variabileGlobale = 10; // Variabile globale all'interno della funzione

  if (true) {
      let variabileLocale = 5; // Variabile locale al blocco if
      console.log(variabileGlobale); // Accessibile all'interno del blocco
  }

  // console.log(variabileLocale); // Errore! La variabile non è visibile qui
}

esempioScope();
```

ereditarietà in JavaScript incapsulamento (*)?

L'ereditarietà in javascript viene implementata attraverso i prototipi, ogni oggetto ha un prototipo al quale si possono aggiungere proprietà a ogni oggetto che eredita da una specifica classe facendo *Classe.prototype.attribute* e assegnando un valore oppure definendone un opportuno constructor(){} . L'incapsulamento non è supportato tant'è che non esistono modificatori di accesso, tuttavia è possibile implementarlo tramite le chiusure/funzioni annidate .

Si dice che javascript è basato sugli oggetti e non orientato ad oggetti: si spieghi quali sono le differenze.

La differenza sostanziale è che altri linguaggi gli oggetti sono istanziati in Classi, il concetto vero e proprio di "classe" non esiste in Javascript e quindi ogni elemento è

considerato oggetto.L'ereditarietà si realizza tramite prototipi,quindi la gerarchia è realizzate tramite istanze e non tramite classi.
Inoltre non realizza altri aspetti tipici della programmazione ad oggetti, come l'incapsulamento.

in JavaScript posso assegnare ad un elemento di un vettore la stringa "diciassette" e ad un altro il numero 17? e in java? in caso di risposta/e affermativa/e, si spieghi come si fa?

In JavaScript è possibile farlo mentre in Java no, questo è dovuto dal fatto che il primo è debolmente tipato mentre il secondo è fortemente tipato.

In Javascript:

```
//viene creato l'array
var = new Array();
//assegno ad ogni elemento dell'array il valore che voglio
var[0]="diciassette";
var[1]=17;
//il seguente codice non produce errore poiché javascript ammette
questo tipo di assegnazioni, una cosa simile in Java avrebbe dato
errore a tempo di compilazione
```

(anche se: JavaScript è debolmente tipizzato e consente di assegnare tipi diversi a elementi di un array senza problemi, mentre Java è fortemente tipizzato e richiede l'uso di oggetti generici (come `Object`) e il casting esplicito per ottenere una funzionalità simile, ma in modo meno flessibile e meno intuitivo.)

localStorage, sessionStorage, delete web storage (web storage)

In HTML5 è stato introdotto il WebStorage, che permette di salvare file, anche di grandezza contenuta, all'interno del browser.
Parliamo di LocalStorage e SessionStorage, i file salvati sono di tipo chiave-valore ; Nel caso del local storage questi valori possono essere mantenuti finché non si decide di eliminarli, nel caso del session storage vengono mantenuti fino alla fine della sessione corrente; sono presenti poi inoltre metodi di delete per appunto pulire il web storage.
Entrambi sono accessibili via javascript facendo semplicemente locaStorage.etc.etc.etc

DOM

Cos'è la DOM? se ne descrivano gli aspetti più importanti. In che linguaggio è implementato?

La DOM (Document Object Model) è un API per l'accesso dei nodi della rappresentazione ad albero delle pagine HTML,XML etc. Esso permette di accedere ai nodi attraverso vari linguaggi,tra cui il più comune è javascript,e di manipolare ed estrapolare il contenuto di essi; permette inoltre di implementare eventi legati a questi nodi.

La DOM è indipendente dalla piattaforma e dal linguaggio utilizzato.

DOM del W3C (*)?

La DOM (Document Object Model) del W3C (World Wide Web Consortium) è uno standard che definisce la rappresentazione ad albero di un documento, come una pagina web HTML o un documento XML, e specifica come i programmatori possono accedervi e manipolarlo. Questa specifica è fondamentale per garantire che la DOM sia implementata in modo coerente tra diversi browser web e ambienti di sviluppo.

La DOM del W3C stabilisce una serie di standard che definiscono i metodi, le proprietà e gli eventi per l'accesso e la manipolazione dei documenti web. Questi standard consentono agli sviluppatori di scrivere codice che funziona in modo simile su diversi browser, indipendentemente dalla piattaforma. Ciò è particolarmente importante per garantire la compatibilità tra browser e la coerenza nelle applicazioni web.

Gestione degli eventi ?

Un evento è una azione dell'utente o generata dal sistema stesso.

Gli elementi della DOM vengono predisposti all'ascolto di questi eventi ed eseguono il codice per gestirli.

Il metodo addEventListener aggiunge un gestore di eventi all'elemento della DOM a cui viene aggiunto.

Il metodo removeEventListener rimuove il gestore di eventi associato all'elemento della DOM.

Cosa rappresenta un nodo nella DOM?

I nodi della DOM sono rappresentati dagli elementi del documento.

I tipi di nodo previsti dalla DOM W3C sono 12,di cui 7 si applicano ai documenti HTML.

Elemento: un nodo che corrisponde all'etichetta associata all'elemento

Attributo : un nodo che corrisponde all'attributo associato all'etichetta

Testo : un nodo che corrisponde ad una porzione di testo presente nel documento

Commento : un nodo che corrisponde ai commenti HTML

Document : un nodo che corrisponde al documento stesso

TipoDocumento : un nodo che corrisponde al DTD

Frammento : un nodo che corrisponde ad elementi esterni alla pagina HT

Che differenza c'è tra parser a DOM e parser a eventi?

Parser a DOM si occupa di effettuare il parsing di un documento XML,HTML a DOM.

Esso carica in memoria tutto il documento XML,HTML e crea un albero che rappresenta il documento.

Parser a Eventi , effettua il parsing di un documento XML,HTML ed è basato su eventi.

Non carica tutto l'albero del documento in memoria.

CMS & Web Framework

cos'è un CMS?A cosa serve?

Con la nascita del web 2.0 le pagine web diventano più dinamiche ed interattive grazie ad elementi quali AJAX,CSS e web frameworks,e riescono ad accorpare dentro di loro una serie di funzioni tant'è che alcune diventano proprio delle Rich Internet Applications (RIA).
Tra le RIA si annoverano i CMS (Content Management System), che sono in sostanza delle piattaforme che permettono a chiunque,senza la scrittura di codice, di accedere a varie funzioni come management pool di utenti, gestione contenuti multimediali, creazione e stilizzazione di pagine web senza scrivere codice.

cos'è un Web Framework?A cosa serve?

Cosa si intende per architettura a tre strati (three tier)?

modello MVC (*)?

Java

Si dice che Java è un linguaggio sicuro: quali sono le caratteristiche che lo rendono tale, in particolare in relazione alle applicazioni web?

Servlet

Ciclo di vita di una servlet?

Dove si scrivono i messaggi di errore in una servlet?

Metodi dei servlet (*)?

L'interfaccia HttpServletRequest e HttpServletResponse?

Esercizio conversione codice html in servlet.

Session Tracking

Cos'è e come funziona il session tracking mediante campi nascosti?

Quali sono le differenze tra session tracking coi campi nascosti e con le variabili di sessione?

Cosa sono, a cosa servono e come funzionano le variabili di sessione

Cosa si intende per "session tracking" e quali sono i modi per affrontarlo?

L'interfaccia HttpServletRequest e HttpServletResponse?

Esercizio conversione codice html in servlet.

Il meccanismo di autenticazione può essere implementato in due modi diversi: il primo sulle caratteristiche dell'HTTP, il secondo no. si spiegano questi due diversi approcci.

per il meccanismo di session tracking mediante campi nascosti occorre supporto dell'HTTP? si spieghi nel dettaglio come funziona

meccanismo session tracking mediante autenticazione

meccanismo session tracking mediante riscrittura dell'url

JSP & Struts2

Cosa sono e come si presentano le JSP?

A proposito di JSP, cosa si intende per "oggetti impliciti"? e quali sono i loro possibili ambiti di definizione (scope)?

Si descriva il funzionamento di una JSP.

Quali sono i modi per introdurre istruzioni Java in una JSP?

Qual è la differenza tra dichiarazioni ed espressioni nelle JSP?

Quali sono le differenze tra scriptlet e dichiarazioni in una JSP?

Cosa sono i Javabeans (java bean)?

WebUML

Fare esercizi e leggere semplicemente capitolo

XML e XHTML

incluso nel programma 2022-2023?

AJAX

Cos'è e per cosa viene usato l'oggetto XMLHTTPRequest?

AJAX si fonda su un certo numero di strumenti, alcuni obbligatori, altri tipicamente utilizzati in questo

framework: si spieghi quali sono e che funzione hanno in AJAX, e cos'è AJAX

BOTS, SPIDERS & CRAWLERS

Quali sono le differenze tra bot, spider e crawler?

Cosa è ed a che serve il file robots.txt?

Servizi Web e Cloud Computing

Cosa si intende per servizio web? In quale/i linguaggio/i si implementa?
Con il termine "servizio web" si vuole intendere un insieme di funzionalità che possono essere innescate
da software e/o hardware. I linguaggi utilizzati possono essere anche più di uno in un stesso servizio web,
però in genere si preferisce utlizzare l'XML.
Cosa è SOAP e a cosa serve? nei servizi web?

Qual è la differenza tra scoperta manuale e autonoma di un servizio web?
Per scoperta (discovery) si intende la localizzazione di una descrizione elaborabile in modo automatico di
un servizio Web che, pur sconosciuto fino a quel momento, risponde a criteri funzionali dati. Per servizio
di scoperta (discovery service) si intende un servizio che aiuta nel processo di scoperta; può essere
implementato sia dall'agente provider che dall'agente requester o da un altro agente.
Cos'è e a cosa serve WSDL?
WSDL permette di descrivere il servizio e rappresenta un accordo che sta alla base dei meccanismi di interazione con quel servizio. Le definizioni dei servizi Web possono poi venir mappate verso un qualsiasi linguaggio di programmazione, piattaforma, modello di oggetti o sistemi di messaggi
Cosa si intende per "Discovery service" nei servizi web?
Per servizio di scoperta (discovery service) si intende un servizio che aiuta nel processo di scoperta; può essere implementato sia dall'agente provider che dall'agente requester o da un altro agente.
Quali sono le differenze tra un servizio web e una servlet?
Quali sono i diversi livelli dei servizi forniti dal cloud?
-Infrastructure as a Service (IaaS) sono i servizi che gestiscono le risorse fisiche e le corrispondenti macchine virtuali, e comprendono i server di elaborazione, la memorizzazione dei dati, i firewall e il gestore dei carichi computazionali;
-Platform as a Service (PaaS) rappresentano l'ambiente di sviluppo che permette di implementare le applicazioni e includono linguaggi di programmazione, librerie, frameworks, editor di mashup, dati strutturati;
-Software as a Service (SaaS) comprendono le applicazioni vere e proprie; sono gestiti dal browser web, comprendono i social networks, le applicazioni di utilità,

quali office, gli strumenti di elaborazione video, etc.; l’utente accede ai servizi di questo livello attraverso un portale web e non deve più preoccuparsi della manutenzione e dell’aggiornamento del software

Cos’è il Cloud Computing?

Il cloud computing comprende un insieme di tecnologie informatiche che permettono di fornire come servizi disponibili su richiesta, quindi per cloud computing si intende l'elaborazione come un servizio.

Semantic Web

Cosa si intende per "web semantico?

Cosa si intende per Liked data, Open data e Linken open data (*)

Cosa si intende per RDF e a cosa serve?